

Relatorio curto - BCC264 TP3

1. Objetivo do trabalho

O objetivo deste trabalho e demonstrar que a comunicacao entre processos continua sendo um problema central em Sistemas Operacionais, mesmo quando ela aparece em uma camada de abstracao mais alta que o socket TCP bruto. No TP2, a comunicacao era observada diretamente por meio de socket, porta, TCP, protocolo proprio e concorrencia no servidor. Neste TP3, o mesmo problema aparece em APIs de aplicacao: REST/JSON, GraphQL, gRPC e WebSocket.

A aplicacao implementada representa um sistema simples de estoque. O estado inicial possui tres itens: cadeira com 5 unidades, mesa com 3 unidades e monitor com 2 unidades. O sistema permite consultar estoque, comprar/reservar itens, cancelar/devolver itens e receber notificacoes de mudanca de estado em tempo real.

O ponto mais importante da implementacao e que as quatro interfaces nao representam quatro sistemas independentes. Todas elas acessam o mesmo nucleo de dominio e a mesma fonte de verdade. Assim, uma compra feita por gRPC altera o mesmo estoque que depois pode ser consultado por REST ou GraphQL e tambem gera evento recebido pelo canal WebSocket.

2. Arquitetura escolhida

A solucao usa a Arquitetura B sugerida pelo enunciado: multiplos servicos, todos compartilhando uma fonte de verdade central. O ambiente e composto por cinco containers:

tp3-domain: servico central de dominio, responsavel pelo estoque compartilhado.

tp3-rest: interface REST/JSON.

tp3-graphql: interface GraphQL.

tp3-grpc: interface gRPC baseada em proto/inventory.proto.

tp3-websocket: canal WebSocket para notificacoes em tempo real.

A comunicacao entre os servicos acontece pela rede interna criada pelo Docker Compose. O servico domain escuta na porta interna 7000 e e usado pelas interfaces externas como camada central de estado e regras de negocio. As portas publicas expostas para teste sao:

Interface	Porta	Endereco
REST	8080	http://localhost:8080
GraphQL	8082	http://localhost:8082/graphql
gRPC	50051	localhost:50051
WebSocket	8081	ws://localhost:8081/ws

A organizacao geral pode ser resumida assim:

```
REST :8080 ----\  
GraphQL :8082 ----> domain :7000 --> eventos SSE --> WebSocket :8081/ws  
gRPC :50051 ---/
```

O WebSocket nao altera o estoque diretamente. Ele assina o fluxo de eventos emitido pelo dominio e repassa esses eventos aos clientes conectados. Com isso, qualquer alteracao feita por REST, GraphQL ou gRPC pode ser observada em tempo real pelo WebSocket.

3. Nucleo de dominio e estado compartilhado

O estado compartilhado do sistema e um mapa em memoria mantido pelo servico domain. Esse mapa contem os nomes dos itens e suas quantidades atuais. Todas as operacoes de compra e cancelamento passam por esse servico central.

O dominio implementa validacoes comuns para todas as interfaces:

- o campo item e obrigatorio;
- o campo quantity deve ser um inteiro positivo;
- itens inexistentes retornam erro;
- compras que excedem o estoque disponivel sao rejeitadas;
- cancelamentos/devolucoes aumentam o estoque;

toda mutacao publica eventos de alteracao de estoque.

Cada interface adiciona um campo origin ao chamar o dominio. Esse campo indica se a alteracao veio de rest, graphql ou grpc. O valor aparece nos eventos publicados, permitindo demonstrar que uma operacao feita por uma interface foi observada por outra.

A consistencia em uma unica instancia e protegida por uma fila local de mutacoes no domain. Essa fila serializa compras e cancelamentos, evitando que duas requisicoes concorrentes modifiquem o estoque ao mesmo tempo de forma inconsistente. O objetivo e impedir, por exemplo, que duas compras simultaneas do ultimo monitor sejam aceitas e deixem o estoque negativo.

4. Interfaces implementadas

4.1 REST/JSON

A interface REST esta exposta na porta 8080 e usa mensagens JSON. Ela implementa os endpoints obrigatorios:

GET /health: verifica se o servico REST e o dominio estao ativos.

GET /items: lista todos os itens.

GET /items/:item: consulta um item especifico.

POST /purchase: realiza compra ou reserva.

POST /cancel: cancela ou devolve unidades ao estoque.

Exemplo de compra via REST:

```
curl -i -X POST http://localhost:8080/purchase \
  -H "content-type: application/json" \
  -d '{"item":"monitor","quantity":1,"clientId":"cli-rest"}'
```

O REST segue um modelo orientado a recursos e endpoints HTTP. Nesse modelo, a semantica da operacao aparece principalmente na combinacao entre metodo HTTP, caminho da URL e payload JSON.

4.2 GraphQL

A interface GraphQL esta exposta em http://localhost:8082/graphql. Ela disponibiliza:

query items;

query item(name);

mutation purchase;

mutation cancel.

Exemplo de consulta:

```
query {
  items {
    name
    stock
  }
}
```

Exemplo de mutacao:

```
mutation {
  purchase(item: "monitor", quantity: 1, clientId: "cli-graphql") {
    success
    item
    remaining
    message
  }
}
```

GraphQL evidencia um modelo de API orientado a schema e consultas flexiveis. Diferentemente do REST, o cliente escolhe quais campos deseja receber na resposta, desde que esses campos existam no schema.

4.3 gRPC

A interface gRPC esta exposta em localhost:50051 e segue o contrato definido em

proto/inventory.proto. O serviço implementado é InventoryService, com quatro métodos:

```
ListItems;  
GetItem;  
Purchase;  
Cancel.
```

O gRPC usa uma abordagem RPC tipada e contratual. Em vez de chamar um recurso HTTP por URL, o cliente chama um método remoto definido em um contrato .proto. Esse contrato especifica nomes de métodos, tipos de requisição e tipos de resposta.

O gRPC também se diferencia por usar Protocol Buffers como formato de serialização e HTTP/2 como base de comunicação. O uso de HTTP/2 permite multiplexar várias chamadas concorrentes sobre uma mesma conexão TCP, reduzindo overhead de conexões e consumo de file descriptors em comparação com o padrão tradicional REST sobre HTTP/1.1.

4.4 WebSocket

O WebSocket está exposto em ws://localhost:8081/ws. Ele cria um canal persistente entre cliente e servidor para notificações em tempo real. Quando o estoque é alterado por REST, GraphQL ou gRPC, o domínio emite eventos e o serviço WebSocket os repassa aos clientes conectados.

Eventos emitidos pelo sistema incluem:

```
purchase;  
cancel;  
stock_update;  
low_stock;  
out_of_stock;  
items_snapshot.
```

Um exemplo de evento recebido via WebSocket é:

```
{  
  "event": "stock_update",  
  "item": "monitor",  
  "stock": 1,  
  "origin": "grpc"  
}
```

Esse canal permite demonstrar o padrão de interação assíncrono e orientado a eventos, diferente do request/response tradicional usado em REST, GraphQL e chamadas unary de gRPC.

5. Comparação conceitual entre as tecnologias

O TP amplia a noção clássica de IPC porque mostra que processos podem se comunicar não apenas por mecanismos locais do sistema operacional, mas também por rede, IP, porta, transporte e protocolos de aplicação. O socket continua existindo por baixo, mas a aplicação passa a usar abstrações mais ricas para organizar as mensagens.

Socket não é a mesma coisa que WebSocket. Socket é uma interface de programação usada pelo processo para abrir um canal de comunicação, normalmente sobre TCP ou UDP. WebSocket é um protocolo de aplicação que inicia a comunicação por HTTP e depois mantém um canal persistente e bidirecional.

HTTP, REST, GraphQL, gRPC e WebSocket ficam na camada de aplicação da pilha conceitual discutida no TP. TCP fica na camada de transporte, IP fica na camada de rede e o socket é a API usada pelo processo para acessar essa comunicação. JSON, Protocol Buffers, schemas e arquivos .proto descrevem representação de dados e contratos.

GraphQL e gRPC não devem ser confundidos com transporte. GraphQL é um modelo de API baseado em schema, queries e mutations, normalmente servido sobre HTTP. gRPC é um modelo de chamada remota tipada, normalmente usando HTTP/2 e Protocol Buffers. O transporte propriamente dito continua abaixo, provido por TCP.

JSON e Protocol Buffers também cumprem papéis diferentes. JSON é textual, legível por humanos e usado neste trabalho em REST e GraphQL. Protocol Buffers é binário, tipado e mais compacto,

usado no gRPC a partir do contrato .proto. JSON favorece simplicidade e depuracao manual; Protocol Buffers favorece contrato forte, eficiencia e interoperabilidade em RPC.

6. Consistencia, concorrencia e experimento

O experimento de concorrencia do projeto esta implementado em scripts/concurrency-test.js. O script prepara o item monitor com uma unidade em estoque e dispara duas compras concorrentes pela interface REST.

O resultado esperado e que uma compra seja aceita e a outra rejeitada por estoque insuficiente. Dessa forma, o estoque final permanece em zero, e nao negativo. Esse comportamento demonstra que, em uma unica instancia, a fila local de mutacoes no dominio e suficiente para serializar as alteracoes de estado.

Comando usado:

```
docker compose exec rest node scripts/concurrency-test.js
```

Em uma unica instancia, uma estrategia local como fila, mutex ou operacao atomica pode funcionar porque todas as mutacoes passam pelo mesmo processo e pela mesma memoria. No entanto, se o sistema fosse executado em multiplas instancias ou em hosts diferentes, um mutex local nao bastaria. Cada processo teria sua propria memoria e sua propria fila. Nesse caso, seria necessario mover a fonte de verdade para um banco transacional, usar operacoes atomicas externas, fila centralizada ou outro mecanismo de coordenacao adequado.

O trabalho nao exige lock distribuido, consenso ou replicacao, mas e importante reconhecer o limite da solucao local. A implementacao atual e coerente para uma instancia unica, que e o escopo pratico do TP.

7. Teste de carga, threads e assincronia

O projeto inclui o script scripts/load-test.sh, que executa um teste simples contra GET /items na interface REST usando hey ou wrk, caso uma dessas ferramentas esteja instalada no host.

Comando:

```
./scripts/load-test.sh
```

O servidor foi implementado em Node.js, que usa um modelo baseado em event loop para I/O de rede. Isso significa que ele consegue lidar com muitas conexoes sem criar uma thread por requisicao. Enquanto as operacoes sao principalmente I/O-bound, esse modelo reduz o custo de troca de contexto em comparacao com modelos que associam uma thread bloqueante a cada conexao.

Conceitualmente, a diferenca entre thread e programacao assincrona aparece nesse ponto. Uma thread representa uma unidade de execucao escalonada pelo sistema operacional. A programacao assincrona permite que o processo inicie operacoes de I/O e continue tratando outros eventos enquanto aguarda respostas. No caso deste TP, as interfaces fazem chamadas de rede ao dominio e nao executam processamento pesado de CPU. Por isso, o modelo assincrono do Node.js e adequado.

Se o servidor recebesse carga muito alta, a vazao dependeria de fatores como latencia das chamadas internas, custo de serializacao, numero de conexoes, limites de CPU e eventuais gargalos na thread principal. Se houvesse processamento intenso de CPU, a thread principal do Node.js poderia se tornar gargalo, exigindo workers, balanceamento ou outra estrategia.

8. Containerizacao e Docker Hub

O projeto foi empacotado com Dockerfile e docker-compose.yml. O Dockerfile usa a imagem base node:22-alpine, instala as dependencias de producao e copia codigo-fonte, contratos, scripts, documentacao e colecao Postman para dentro da imagem.

O docker-compose.yml instancia os cinco servicos do projeto e expoe as portas necessarias para teste. A mesma imagem Docker e usada para todos os containers; o que muda entre eles e o comando de inicializacao. Isso evita duplicacao de imagens e mantem a entrega simples.

A imagem publicada no Docker Hub para submissao e:

```
gustavomeira1/tp3so:latest
```

Comandos minimos para reproducao pelo avaliador:

```
docker pull gustavomeira1/tp3so:latest  
DOCKERHUB_IMAGE=gustavomeira1/tp3so:latest docker compose up
```

Comandos usados para reconstruir e publicar uma nova versao:

```
docker login  
docker build -t gustavomeira1/tp3so:latest .  
docker push gustavomeira1/tp3so:latest
```

9. Demonstracao prevista

A demonstracao em video deve seguir a sequencia:

- Subir os containers com Docker Compose.
- Mostrar o REST listando itens e realizando uma compra.
- Mostrar o GraphQL consultando itens e executando mutation.
- Mostrar o gRPC chamando ListItems, Purchase, GetItem e Cancel.
- Abrir o cliente WebSocket e observar notificacoes de estoque.
- Realizar uma alteracao por uma interface e consultar o resultado por outra.
- Executar o teste de concorrencia e explicar por que apenas uma compra deve vencer.
- Mostrar o nome da imagem publicada no Docker Hub.

Essa demonstracao evidencia que REST, GraphQL, gRPC e WebSocket sao diferentes modelos de comunicacao de aplicacao sobre o mesmo problema central: processos trocando informacao de forma organizada, consistente e observavel.